

PCD - Programação Concorrente e Distribuída

Threads: processos ligeiros

Luís Mota

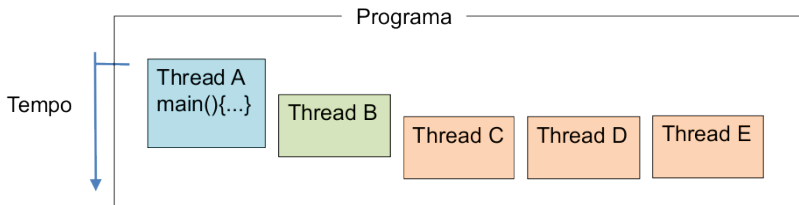
September 23, 2024

Estrutura

- 1 Enquadramento
- 2 Gestão de memória
- 3 Execução
- 4 Aspetos práticos

O que é uma *Thread*

- Programação concorrente funciona com base em *threads*
- Em português, estas designam-se *processos ligeiros*
- Termo inglês vem de *Thread of control*, i.e., linha de controlo
- Código que corre de uma forma concorrente



Porquê usar várias *Threads*?

- Efetuar várias tarefas simultaneamente
- Não bloquear a aplicação à espera de execução de operações lentas:
 - *Input* do utilizador
 - Leitura e escrita em ficheiros
 - Envio ou receção de dados de outras aplicações
- Tirar partido dos vários núcleos (*cores*) do CPU

Processos vs. *threads*

- Processos (ou programas, ou aplicações...):
 - Cada um tem o seu espaço de endereçamento de memória
 - Não partilham memória nem outros recursos próprios com outros processos
 - Comunicação entre processos terá de ser feita por mensagens ou algo semelhante
- Processos ligeiros (ou *threads*):
 - Existem em paralelo dentro de um mesmo processo
 - Partilham o espaço de endereçamento de memória do seu processo, pelo que podem aceder às mesmas variáveis
 - Existe facilidade de comunicação entre *threads* do mesmo processo, mas interferências têm de ser evitadas

Estrutura

- 1 Enquadramento
- 2 Gestão de memória
- 3 Execução
- 4 Aspetos práticos

Partilha de Memória I

Aplicação multi-thread

Memória

Thread 1

PC &
Stack

Thread 2

PC &
Stack

Thread 3

PC &
Stack

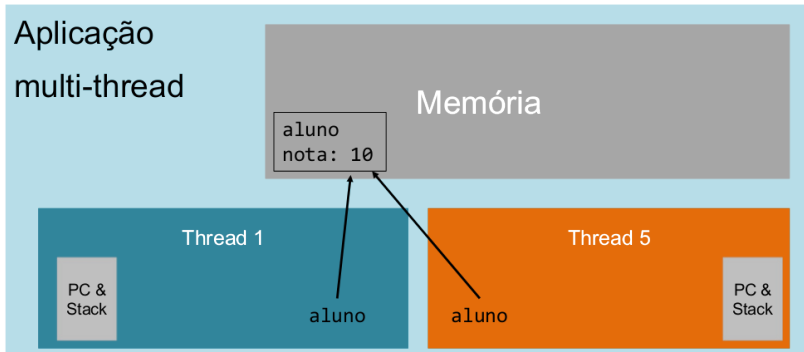
Thread 4

PC &
Stack

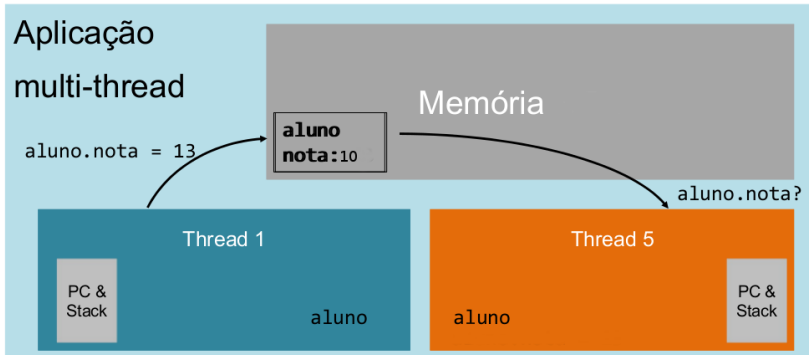
Thread 5

PC &
Stack

Partilha de Memória II



Partilha de Memória III



Estrutura

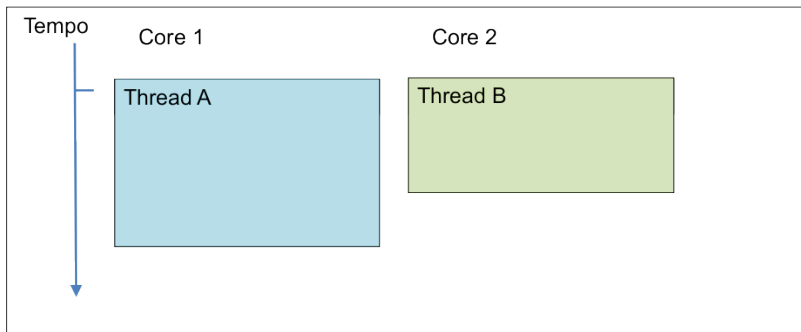
- 1 Enquadramento
- 2 Gestão de memória
- 3 Execução**
- 4 Aspetos práticos

Como é gerida a execução de várias threads?

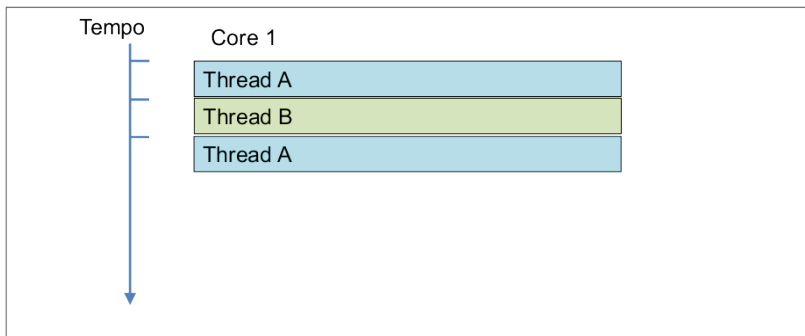
Diferentes cenários para a execução:

- Paralela
- Intercalada
- Mista

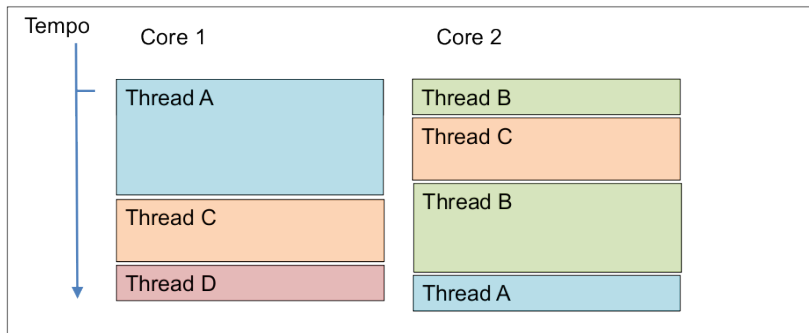
Execução em paralelo



Execução intercalada



Execução mista



Ações Atómicas

- Uma *thread* é uma sequência de ações (instruções)
- Ações complexas podem ser decompostas em ações mais simples
- As ações que não podem ser decompostas são **atómicas**
- Uma *thread* apenas pode ser interrompida entre ações atómicas.

Paradigma de programação concorrente

- em qualquer instante uma *thread* pode:
 - 1 (re)começar a ser executada;
 - 2 deixar de ser executada entre ações atómicas
- o programador não controla em detalhe a execução das *threads*

Cenário com 2 *threads*

- A ordem por que são executadas as ações das diferentes *threads* pode variar de execução para execução
- Os resultados podem ser diferentes de cada vez que é executado o programa

Exemplo de 2 threads

Thread A

```
println("Sou a thread A")  
println("Adeus - A")
```

Thread B

```
println("Sou a thread B")  
println("Adeus - B")
```

Output:

```
Sou a thread A  
Adeus - A  
Sou a thread B  
Adeus - B
```

```
Sou a thread A  
Sou a thread B  
Adeus - A  
Adeus - B
```

```
Sou a thread A  
Sou a thread B  
Adeus - B  
Adeus - A
```

...

```
Sou a thread B  
Adeus - B  
Sou a thread A  
Adeus - A
```

Estrutura

- 1 Enquadramento
- 2 Gestão de memória
- 3 Execução
- 4 **Aspetos práticos**

Lançamento e execução

- Em Java as *threads* são representadas pela classe `Thread`
- Iniciam a execução após invocação do método `start()`
- Quando iniciada, a *thread* vai executar o método `run()`
- Nunca confundir `start` com `run`, apesar de o significado das palavras ter semelhanças!
- O programador normalmente redefine o `run` e apenas invoca o `start`!

Exemplo de *thread* em Java

```
public class MyThread extends Thread {  
    @Override  
    public void run() {  
        System.out.println(getName());  
    }  
    public static void main(String[] args) {  
        MyThread m = new MyThread();  
        m.start();  
    }  
}
```

Multithreading em Java

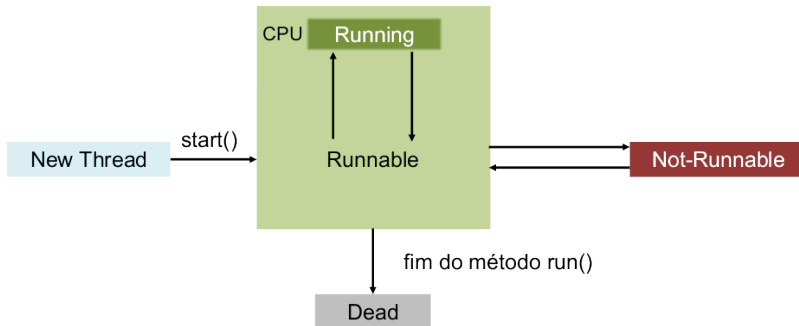
- *Thread* inicial é criada automaticamente e corre o método `main()`
- Restantes *threads* são iniciadas pelo programador
- As *threads* correm de uma forma concorrente
- Uma *thread* termina quando tiver chegado ao fim do seu método `run`. Uma aplicação termina quando todas as suas *threads* tiverem terminado.

Multithreading em Java

```
public class Main {  
    public static void main(String[] args) {  
        MyThread t1= new MyThread("T1");  
        t1.start();  
        ...  
    }  
}
```

```
public class MyThread extends Thread {  
    ...  
    @Override  
    public void run() {  
        System.out.println(getName());  
        ...  
    }  
}
```

Ciclo de vida das *Threads*



Pausar uma *thread* (sleep)

```
static void Thread.sleep(time) throws  
InterruptedException
```

Método de classe (static) que pausa a execução da *thread* que está a correr durante um tempo definido.

Interrupção de *threads*

- Mecanismo para indicar a uma *thread* que deve interromper o que está a fazer.
- Uma *thread* pode interromper outra usando o método `interrupt`.
- O programador escolhe como reagir à interrupção.
- Não é forçoso que a execução acabe: a interrupção pode até ser ignorada

Exemplo de interrupção e pausa

```
public class MyThread extends Thread {  
    public void run() {  
        try {  
            while(true){  
                System.out.println(currentThread()+": sleeping.");  
                sleep(4000);  
            }  
        } catch (InterruptedException e) {  
            System.out.println(currentThread()+": Interrupted!");  
        }  
    }  
    public static void main(String args[])  
        throws InterruptedException {  
        Thread t = new MyThread ();  
        t.start();  
        Thread.sleep(12000);  
        t.interrupt();  
    }  
}
```

Esperar que uma *thread* termine

```
void join() throws InterruptedException
```

Espera que uma *thread* termine. Deve, naturalmente, ser chamado a partir de outra *thread*...

Exemplo join

```
public class MyThread extends Thread {  
    public void run() {  
        try {  
            for(int i = 0; i < 10; i++){  
                System.out.println(currentThread() + ' ' + i);  
                sleep(2000);  
            }  
        } catch (InterruptedException e) {}  
    }  
    public static void main(String args[])  
        throws InterruptedException {  
        Thread t = new MyThread ();  
        t.start();  
        try {  
            t.join();  
        } catch (InterruptedException e) {}  
        System.out.println("Main done, all done!");  
    }  
}
```

Interface Runnable

- É uma alternativa necessária quando se quiser fazer que uma classe seja `Thread` e estenda outra classe qualquer (pois o Java não permite que uma classe herde de duas classes).
- Apenas define um método: `void run()`
- Para executar, enquanto *Thread*, uma instância de uma classe `MyRunnable` que implemente o interface `Runnable`, teríamos p.ex.:

```
MyRunnable mr=new MyRunnable();  
new Thread(mr).start();
```

- 1 Enquadramento
- 2 Gestão de memória
- 3 Execução
- 4 Aspetos práticos